UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

Séptimo semestre, paralelo A

Notas de la presentación: justificación del diseño

Taller de modelado de comunicación entre procesos distribuidos

Código de la actividad: GA-TL-02 - FORM

Escenario 1: AeroAndes Reservas

Aplicaciones Distribuidas (ISR-701)

Docente: Gleiston Cicerón Guerrero Ulloa, PhD

Integrantes

Herrera Barco Humberto Aldair	(zona A)
Toaquiza Zambrano Belinda Betzabeth	(zona B)
Piguave Montaño Nixon Antonio	(zona C)
Emanuel Pino Juliana Romina	(zona D)

Quevedo, Ecuador — 11 de noviembre de 2026

1. Decisiones del grupo

Comunicación entre reservas y pagos

Elección. Se eligió una llamada RPC síncrona de Reservas a Pagos (gRPC sobre HTTP/2). Pagos responde CONFIRMADO, RECHAZADO o DESCONOCIDO, y la reserva queda en estado PENDIENTE_PAGO mientras tanto.

Latencia. La llamada directa evita la espera adicional de pasar por un broker y entrega el resultado del pago en cuanto la pasarela responde. Como la pasarela puede tardar hasta 30 s, el cliente ve el estado PENDIENTE_PAGO mientras espera y no queda bloqueado sin información (R4). RPC se diseñó para ofrecer una semántica simple y una comunicación rápida [1]. En pruebas con microservicios, gRPC tuvo menor tiempo de respuesta que REST y GraphQL [2].

Volumen de mensajes. Es un mensaje corto por cada intento de pago. El enunciado no fija una tasa de pagos, así que se dimensiona con la carga de reservas. gRPC reutiliza una conexión HTTP/2 y soporta varias llamadas simultáneas sobre ella.

Necesidad de respuesta inmediata. Necesita respuesta inmediata: Reservas espera el resultado del pago, y en vuelos los usuarios son sensibles al tiempo de espera. Se descartó RabbitMQ en este enlace porque ordena mejor los reintentos, pero añade un salto y retrasa el resultado.

Tolerancia a fallos. Si la pasarela no responde (F1), la llamada vence al superar el plazo máximo de 30 s (R4). El pago queda en DESCONOCIDO, se libera el asiento y, si el cobro sí ocurrió, se reembolsa como compensación (ver la decisión sobre F1, F2 y F3).

Consistencia. El pago debe ejecutarse a lo sumo una vez. Se logra enviando una clave de idempotencia en cada llamada, de modo que un reintento se detecte como duplicado [3].

Fuentes. [1], [2], [3]

Comunicación entre embarque y reservas, y operación sin conexión

Elección. Se eligió gRPC entre el servidor de embarque y Reservas. Los lectores y quioscos siguen comunicándose con el servidor de embarque por sockets TCP, que es la interacción obligatoria con sockets. El servidor de embarque guarda el manifiesto del vuelo para poder operar sin red.

Latencia. R6 exige responder en menos de 1 s. gRPC y Thrift resultaron más rápidos que REST entre microservicios, gracias a su velocidad y a la compresión de los mensajes [4]. Eso respalda gRPC para validar pases rápidamente, en línea con lo medido en [2]. Se descartó REST entre embarque y reservas por esa mayor latencia, y UDP en los lectores porque un mensaje perdido dejaría al pasajero sin respuesta.

Volumen de mensajes. R6 indica unos 40 lectores enviando mensajes cortos de formato fijo. Es un volumen bajo por lector, que una conexión persistente soporta sin dificultad.

Necesidad de respuesta inmediata. Necesita respuesta inmediata mientras haya red, porque el pase se valida en el momento del abordaje. Sin red, la validación se resuelve localmente y la sincronización queda diferida.

Tolerancia a fallos. Si se corta la red del aeropuerto (F3), el servidor valida contra el manifiesto local, guarda cada abordaje en una cola y los envía a Reservas cuando la conexión vuelve. Esto sigue la estrategia de continuar operando durante la partición y procesar después las acciones encoladas [5].

Consistencia. Se acepta consistencia eventual con riesgo acotado. Brewer señala que los cajeros automáticos eligen disponibilidad con consistencia débil, pero con riesgo limitado [6]. Aquí el riesgo se acota porque el manifiesto local contiene solo los pasajeros del vuelo, y las discrepancias se reconcilian al

reconectar.

Fuentes. [4], [5], [6]

Control de asientos y retención

Elección. Reservas pide la retención al inventario por gRPC. Se eligió un bloqueo corto en la base de datos, aplicado solo al cambiar el estado del asiento, combinado con una retención lógica con vencimiento de 10 minutos. En el diagrama corresponden los mensajes `retenerSiLibre`, `confirmarSiRetencionVigente` y `liberarSiRetenidaPor`.

Latencia. El bloqueo dura lo que tarda una actualización condicional, no los 10 minutos de la compra, por lo que no frena a otros clientes. Mantener un registro bloqueado mientras el usuario completa su compra es riesgoso: si se pierde la comunicación, el registro queda bloqueado. Por eso se recomienda verificar la versión de la fila en lugar de conservar el bloqueo, y el ejemplo típico de ese trabajo es la reserva de pasajes de vuelo [7].

Volumen de mensajes. Todas las retenciones y confirmaciones se dirigen al servicio de inventario, que es el único dueño del estado de los asientos. El enunciado no fija una tasa de reservas por segundo.

Necesidad de respuesta inmediata. Necesita respuesta inmediata: retener un asiento o rechazar la solicitud debe resolverse en el momento, para que el cliente sepa si pudo reservar.

Tolerancia a fallos. Si el cliente o el pago se caen a mitad de la compra, la retención vence sola a los 10 minutos (R2) y el asiento se libera. Es la idea de un lease, un derecho con plazo en el que las fallas afectan el rendimiento pero no la corrección [8].

Consistencia. Un asiento nunca debe venderse dos veces (R3). Se garantiza con una única escritura atómica y condicional sobre el estado del asiento, que solo avanza si el asiento sigue libre o retenido por la misma reserva. El primero en llegar gana.

Fuentes. [7], [8]

Uso de la caché de disponibilidad

Elección. Se eligió usar la caché solo para las búsquedas de disponibilidad. Retener y confirmar asientos van siempre al servicio de inventario, y este invalida la entrada de caché cuando un asiento se vende o se libera (mensajes `invalidar` del diagrama).

Latencia. R1 exige 1 200 consultas por segundo con el 95 % por debajo de 300 ms. Consultar la base de datos en cada búsqueda no sostendría esa carga, mientras que la caché responde desde memoria. Se descartó consultar siempre la base de datos por esa razón, y también confirmar compras con datos de la caché, porque podría vender dos veces el mismo asiento (R3).

Volumen de mensajes. Se esperan 1 200 consultas por segundo (R1). Las lecturas se sirven desde la caché, y solo las compras llegan al inventario y a la base de datos.

Necesidad de respuesta inmediata. La búsqueda necesita respuesta inmediata, pero tolera datos ligeramente desactualizados: puede mostrar libre un asiento que ya se vendió. Por eso la compra no se apoya en la caché.

Tolerancia a fallos. Si la caché falla, las búsquedas pasan a la base de datos con mayor latencia, pero el sistema sigue funcionando. Nunca se vende un asiento por un dato viejo, porque la compra siempre se valida en el inventario.

Consistencia. La caché ofrece consistencia débil solo para lectura. Expirar entradas por tiempo devuelve documentos desactualizados, mientras que la invalidación mantiene consistencia fuerte con un costo

similar [9]. Con el benchmark de comercio web TPC-W, la invalidación resultó la más adecuada para comercio electrónico [10].

Fuentes. [9], [10]

Respuesta conjunta a los fallos F1, F2 y F3

Elección. Para F1 se usa un timeout con liberación del asiento y compensación. Para F2 se usa una cola durable en RabbitMQ con reintento. Para F3 se usa un manifiesto local en el servidor de embarque con sincronización posterior.

Latencia. Ninguna de las tres respuestas añade espera al camino normal: cada una actúa solo cuando algo falla. El timeout de 30 s de F1 acota la espera del cliente (R4).

Volumen de mensajes. Las confirmaciones de reserva viajan por la cola hacia Notificaciones (R5), un mensaje por reserva confirmada, que debe llegar en menos de 1 minuto después del pago. El enunciado no fija una tasa para ese flujo. RabbitMQ se puede configurar con nodos agrupados y colas replicadas para ganar disponibilidad, aunque eso compite con el rendimiento [11].

Necesidad de respuesta inmediata. F1 es inmediata: el cliente recibe el estado pendiente sin quedar bloqueado. F2 y F3 son diferidas: la notificación se entrega cuando el servicio vuelve, y los abordajes se envían al recuperar la red.

Tolerancia a fallos. F1: una transacción larga se divide en pasos y, si uno falla, se ejecutan transacciones compensatorias para deshacer los anteriores [12]. En microservicios este patrón no aísla las lecturas de estados incompletos [13], por eso la reserva permanece en PENDIENTE_PAGO y no se muestra como vendida. F2: la confirmación espera en la cola y se reintenta hasta que Notificaciones responde. F3: se aplica lo descrito en la decisión de embarque.

Consistencia. F1 exige que el cobro ocurra una sola vez. Lograrlo ante caídas es difícil, porque cliente y servidor deben registrar sus acciones en almacenamiento estable [14]. F2 y F3 aceptan consistencia eventual, con las garantías y límites que describen [15].

Fuentes. [11], [12], [13], [14], [15]

2. Zona A: Clientes y sistemas externos

Redactado por: Herrera Barco Humberto Aldair

[PENDIENTE: nombre de la decisión 1]

Elección. [PENDIENTE: qué se eligió, indicando el enlace o mensaje del diagrama]

Latencia. [PENDIENTE: por qué conviene en latencia (citar requisito R)]

Volumen de mensajes. [PENDIENTE: cuántos mensajes se esperan y cómo los soporta]

Necesidad de respuesta inmediata. [PENDIENTE: si necesita respuesta inmediata o puede ser diferida, y por qué]

Tolerancia a fallos. [PENDIENTE: qué pasa si falla (citar F)]

Consistencia. [PENDIENTE: qué garantía de consistencia se necesita y cómo se logra]

Fuentes. [PENDIENTE: fuente que respalda la decisión, con `\cite{clave}`]

[PENDIENTE: nombre de la decisión 2]

Elección. **[PENDIENTE: qué se eligió, indicando el enlace o mensaje del diagrama]**

Latencia. **[PENDIENTE: por qué conviene en latencia (citar requisito R)]**

Volumen de mensajes. **[PENDIENTE: cuántos mensajes se esperan y cómo los soporta]**

Necesidad de respuesta inmediata. **[PENDIENTE: si necesita respuesta inmediata o puede ser diferida, y por qué]**

Tolerancia a fallos. **[PENDIENTE: qué pasa si falla (citar F)]**

Consistencia. **[PENDIENTE: qué garantía de consistencia se necesita y cómo se logra]**

Fuentes. **[PENDIENTE: fuente que respalda la decisión, con `\cite{clave}`]**

[PENDIENTE: nombre de la decisión 3]

Elección. **[PENDIENTE: qué se eligió, indicando el enlace o mensaje del diagrama]**

Latencia. **[PENDIENTE: por qué conviene en latencia (citar requisito R)]**

Volumen de mensajes. **[PENDIENTE: cuántos mensajes se esperan y cómo los soporta]**

Necesidad de respuesta inmediata. **[PENDIENTE: si necesita respuesta inmediata o puede ser diferida, y por qué]**

Tolerancia a fallos. **[PENDIENTE: qué pasa si falla (citar F)]**

Consistencia. **[PENDIENTE: qué garantía de consistencia se necesita y cómo se logra]**

Fuentes. **[PENDIENTE: fuente que respalda la decisión, con `\cite{clave}`]**

3. Zona B: Reservas, inventario y datos

Redactado por: Toaquiza Zambrano Belinda Betzabeth

Comunicación entre reservas e inventario

Elección. Se eligió una llamada RPC síncrona con gRPC sobre HTTP/2 entre `reservas` e `inventario`. Por este enlace viajan la consulta de disponibilidad cuando la caché no tiene el dato (mensaje 1.2), la solicitud de retención (mensaje 2.1) y, desde la zona C, la confirmación y la liberación del asiento. Es la interacción RPC obligatoria de la guía. La respuesta va dentro del mismo mensaje (por ejemplo, `resultado := retenerAsiento(...)`) y cada llamada tiene un plazo máximo de espera.

Latencia. R1 exige que el 95 % de las consultas se responda en menos de 300 ms. Solo las búsquedas con fallo de caché llegan al inventario, y para ellas la llamada directa es el camino más corto. En una comparación entre REST, GraphQL y gRPC en microservicios, gRPC tuvo el menor tiempo de respuesta [2], y RPC se concibió para ofrecer una semántica simple y una comunicación rápida [1]. Se descartó REST por esa mayor latencia, y RabbitMQ porque retener un asiento no puede esperar en una cola.

Volumen de mensajes. R1 plantea hasta 1 200 consultas por segundo, pero la caché absorbe las búsquedas con acierto (véase la decisión sobre la caché en la sección 1). El inventario recibe solo los fallos de caché y las operaciones de retención y confirmación, que son una por intento de compra; el enunciado no fija esa tasa. gRPC usa una conexión HTTP/2 y atiende varias llamadas a la vez sobre ella.

Necesidad de respuesta inmediata. Sí la necesita. Reservas debe saber en el momento si el asiento quedó retenido o ya no está disponible para responder al pasajero, y no puede continuar con el pago mientras esa respuesta no llegue.

Tolerancia a fallos. Si el inventario no responde dentro del plazo, la llamada falla y reservas no continúa con el pago: nunca se da por retenido un asiento sin confirmación. Cada llamada lleva `reservaId`, de modo que un reintento no genera una segunda retención; evitar que una misma petición se ejecute más de una vez es el objetivo de las semánticas de a lo sumo una vez [3]. Si el inventario está caído, las búsquedas con acierto en caché siguen respondiendo (R1), pero no se retiene ni se confirma ningún asiento.

Consistencia. Se necesita consistencia fuerte (R3). El inventario es el único servicio que decide el estado de los asientos, y reservas solo actúa sobre lo que este le responde. No hay copias del estado de un asiento que puedan contradecirse, salvo la caché, que se usa únicamente para consultar.

Fuentes. [1], [2], [3]

Registro de la reserva y su estado en la base de datos

Elección. Reservas guarda cada reserva y su estado en la base de datos de reservas (PostgreSQL sobre TCP) con escrituras síncronas. La reserva se crea en PENDIENTE_PAGO (mensaje 2.2) y luego pasa a CONFIRMADA o a CANCELADA según el resultado del pago y la respuesta del inventario. Si el cobro se aprobó pero el asiento ya no se pudo confirmar, queda en REEMBOLSO_PENDIENTE. Los asientos y las reservas están en la misma base de datos, y cada servicio escribe solo en sus propias tablas.

Latencia. Cada escritura es una transacción corta sobre un solo registro, así que suma poco a la compra y no toca el camino de búsqueda de R1, que se sirve de la caché. La reserva se registra antes de pasar al cobro porque el cliente ve el estado PENDIENTE_PAGO mientras la pasarela responde (R4).

Volumen de mensajes. Se hace una escritura por cada cambio de estado de una reserva, un volumen mucho menor que el de consultas de R1. El enunciado no fija la tasa de compras.

Necesidad de respuesta inmediata. Sí para las escrituras que definen el estado: reservas no debe seguir adelante si no logró registrarlo. En cambio, el envío de la confirmación al pasajero es diferido (véase la decisión siguiente).

Tolerancia a fallos. Si reservas se cae después de retener el asiento y antes de registrar la reserva, nada queda bloqueado: la retención vence sola a los 10 minutos (R2). Es la idea de un *lease*, un derecho con plazo en el que las fallas afectan el rendimiento pero no la corrección [8]. Además, ninguna transacción queda abierta mientras el pasajero paga, porque un registro bloqueado durante la compra queda atrapado si se pierde la comunicación [7]. Si la base de datos no está disponible, no se retiene ni se confirma, porque no se acepta vender un asiento sin dejarlo registrado.

Consistencia. Es consistencia fuerte (R3). El estado del asiento y el de la reserva están en la misma base de datos, y cada cambio es una transacción atómica. El evento de confirmación se guarda en la misma transacción que la reserva. La protección contra la doble venta se explica en la decisión del grupo sobre el control de asientos (sección 1).

Fuentes. [7], [8]

Publicación de la confirmación hacia notificaciones

Elección. Al confirmar la reserva, reservas guarda el evento `ReservaConfirmada` en la base de datos, dentro de la misma transacción. Después, un ciclo (mensaje 4) lee los eventos pendientes (mensaje 4.1), los publica en RabbitMQ por AMQP como mensajes persistentes (mensaje 4.2) y los marca como publicados (mensaje 4.3). Es el patrón de bandeja de salida (*transactional outbox*).

Latencia. R5 pide que la confirmación llegue en menos de 1 minuto después del pago y permite enviarla después de responder al cliente. El ciclo de publicación solo suma su propio intervalo, que debe ser mucho menor que ese minuto. Se descartó publicar directamente en RabbitMQ dentro del flujo de compra, porque haría esperar al cliente por un servicio que no hace falta para responderle.

Volumen de mensajes. Se genera un evento por cada reserva confirmada; el enunciado no fija esa tasa. Es un volumen mucho menor que el de las consultas de R1.

Necesidad de respuesta inmediata. No la necesita. El pasajero recibe su respuesta cuando la reserva se confirma, y la notificación se entrega después, de forma asíncrona (R5).

Tolerancia a fallos. F2: si notificaciones está caído, la confirmación no se pierde, porque el evento queda en la base de datos hasta publicarse y luego en una cola durable hasta que notificaciones lo procese. Si reservas se cae entre publicar y marcar, el evento se publica otra vez, así que notificaciones debe tolerar duplicados. Lograr exactamente una vez ante caídas es difícil, porque cliente y servidor deben registrar sus acciones en almacenamiento estable [14]; por eso se acepta entrega al menos una vez con un receptor idempotente. RabbitMQ se ha evaluado en arquitecturas agrupadas de alta disponibilidad y tolerantes a fallos [11].

Consistencia. Se acepta consistencia eventual en este flujo: el estado de la reserva ya es consistente en la base de datos, y la notificación puede llegar con retraso, con los límites que eso implica [15].

Fuentes. [11], [14], [15]

4. Zona C: Pagos y notificaciones

Redactado por: Piguave Montaño Nixon Antonio

[PENDIENTE: nombre de la decisión 1]

Elección. **[PENDIENTE: qué se eligió, indicando el enlace o mensaje del diagrama]**

Latencia. **[PENDIENTE: por qué conviene en latencia (citar requisito R)]**

Volumen de mensajes. **[PENDIENTE: cuántos mensajes se esperan y cómo los soporta]**

Necesidad de respuesta inmediata. **[PENDIENTE: si necesita respuesta inmediata o puede ser diferida, y por qué]**

Tolerancia a fallos. **[PENDIENTE: qué pasa si falla (citar F)]**

Consistencia. **[PENDIENTE: qué garantía de consistencia se necesita y cómo se logra]**

Fuentes. **[PENDIENTE: fuente que respalda la decisión, con `\cite{clave}`]**

[PENDIENTE: nombre de la decisión 2]

Elección. **[PENDIENTE: qué se eligió, indicando el enlace o mensaje del diagrama]**

Latencia. **[PENDIENTE: por qué conviene en latencia (citar requisito R)]**

Volumen de mensajes. **[PENDIENTE: cuántos mensajes se esperan y cómo los soporta]**

Necesidad de respuesta inmediata. **[PENDIENTE: si necesita respuesta inmediata o puede ser diferida, y por qué]**

Tolerancia a fallos. **[PENDIENTE: qué pasa si falla (citar F)]**

Consistencia. **[PENDIENTE: qué garantía de consistencia se necesita y cómo se logra]**

Fuentes. **[PENDIENTE: fuente que respalda la decisión, con `\cite{clave}`]**

[PENDIENTE: nombre de la decisión 3]

Elección. **[PENDIENTE: qué se eligió, indicando el enlace o mensaje del diagrama]**

Latencia. **[PENDIENTE: por qué conviene en latencia (citar requisito R)]**

Volumen de mensajes. **[PENDIENTE: cuántos mensajes se esperan y cómo los soporta]**

Necesidad de respuesta inmediata. **[PENDIENTE: si necesita respuesta inmediata o puede ser diferida, y por qué]**

Tolerancia a fallos. **[PENDIENTE: qué pasa si falla (citar F)]**

Consistencia. **[PENDIENTE: qué garantía de consistencia se necesita y cómo se logra]**

Fuentes. **[PENDIENTE: fuente que respalda la decisión, con `\cite{clave}`]**

5. Zona D: Aeropuerto

Redactado por: Emanuel Pino Juliana Romina

Lectores y quioscos mediante sockets TCP

Elección. Se mantienen sockets TCP persistentes hacia el servidor de embarque. El mensaje 5 envía `solicitudId, vuelo, pase` y devuelve `permitido, motivo`. Los quioscos usan la misma operación como alternativa a los lectores. TCP entrega un flujo ordenado de bytes; la aplicación debe separar los mensajes mediante un formato fijo o una longitud explícita [16].

Latencia. R6 pide validar en menos de 1 s. Reutilizar la conexión evita abrirla con cada lectura. Ese segundo incluye la red local, la validación y la respuesta; es un objetivo que debe medirse, no una garantía de TCP.

Volumen de mensajes. Se consideran unos 40 lectores (R6), con una solicitud corta y una respuesta por lectura. No se especifican lecturas por segundo ni cantidad de quioscos; no se confunden 40 dispositivos con 40 mensajes por segundo.

Necesidad de respuesta inmediata. La respuesta es inmediata porque el personal necesita saber si permite el abordaje. Si no llega una respuesta válida, el lector no debe mostrar autorización.

Tolerancia a fallos. Si se pierde la conexión TCP, el dispositivo reconecta y reintenta con la misma `solicitudId`. TCP no garantiza que la aplicación haya registrado el abordaje. Ante F3, los lectores continúan si la red local y el servidor funcionan; el manifiesto no resuelve una caída de esa red local.

Consistencia. El servidor centraliza las lecturas de las puertas y comprueba el estado del pase antes de registrar el abordaje. La comprobación y el registro deben ser atómicos para impedir que dos lectores autoricen el mismo pase. Repetir una solicitud devuelve su resultado previo. Liskov y sus colaboradores estudian la detección de mensajes duplicados para lograr ejecución a lo sumo una vez [3]. Ese trabajo respalda la necesidad de controlar duplicados; aquí se conservan identificadores y resultados, sin adoptar su protocolo basado en relojes sincronizados.

Fuentes. [3], [16]

Validación en línea con Reservas mediante gRPC

Elección. Se mantiene gRPC sobre HTTP/2 entre Embarque y Reservas. En 5.1, la operación `validarYRegistrarAbordaje` recibe `solicitudId`, `vuelo`, `pase` y devuelve `permitido`, `motivo`. Se usa una llamada unaria: una solicitud y una respuesta [17], [18]. La autorización incluye el registro para evitar separar ambas acciones en llamadas independientes.

Latencia. Para cumplir R6, la llamada lleva un plazo que deja margen, dentro de 1 s, para la red local y la respuesta al lector. gRPC permite configurar ese plazo, pero no lo fija por defecto [19]. Niswar y sus colaboradores obtuvieron menor tiempo de respuesta con gRPC que con REST y GraphQL en sus pruebas de consulta de datos [2]. La caracterización de protocolos de comunicación de Kumar y sus colaboradores aporta otro antecedente de evaluación en microservicios [4]. Estos estudios orientan la elección, pero no demuestran que AeroAndes cumpla R6: se debe medir su recorrido completo antes de fijar el plazo.

Volumen de mensajes. Cada validación en línea genera una solicitud y una respuesta a Reservas. El servidor reutiliza el canal; la descarga de manifiestos y la sincronización se realizan aparte. Los 1 200 accesos por segundo de R1 corresponden a búsquedas, no a la tasa de embarque.

Necesidad de respuesta inmediata. Se espera la autorización de Reservas antes de responder al lector. Un vencimiento del plazo significa que el resultado es desconocido: la operación pudo ejecutarse aunque la respuesta no llegara.

Tolerancia a fallos. Ante F3 o un plazo vencido, se utiliza el manifiesto previamente descargado y se conserva la misma `solicitudId`. Reservas debe reconocer los reintentos para no registrar otra vez una operación ya aplicada. Si no hay manifiesto válido para ese vuelo, se requiere revisión del personal.

Consistencia. Reservas comprueba y registra una sola vez cada pase del vuelo mediante una operación atómica e idempotente. El abordaje no vende ni libera asientos: R2 y R3 siguen bajo Inventario, y los pagos y avisos de R4 y R5 conservan sus flujos existentes. gRPC transporta la llamada; la consistencia depende de estas reglas de la aplicación.

Fuentes. [2], [4], [17], [18], [19]

Manifiesto local y recuperación de F3

Elección. Antes del embarque, 5.2 descarga de Reservas el manifiesto del vuelo con su versión y 5.2.1 lo guarda localmente. Durante F3, 5.3 valida el pase y registra el abordaje pendiente en almacenamiento local persistente. Al volver la conexión, 5.4 envía lotes por gRPC y 5.4.1 marca solo los registros confirmados. No se introduce RabbitMQ en el aeropuerto.

Latencia. La consulta local evita esperar a la red externa y permite buscar el objetivo de menos de 1 s de R6. La recuperación se ejecuta en segundo plano para no retrasar nuevas lecturas. El tiempo total de sincronización depende de la duración del corte y de los pendientes acumulados.

Volumen de mensajes. Se descarga un manifiesto por vuelo y se actualiza antes de operar sin red. Cada abordaje aceptado sin conexión añade un registro pendiente; al reconectar se agrupan en lotes acotados. No se fija un tamaño ni una tasa que el enunciado no proporciona.

Necesidad de respuesta inmediata. La validación local responde inmediatamente; la actualización de Reservas puede esperar hasta la recuperación de F3. Solo se confirma al lector después de guardar de forma durable el abordaje y su registro pendiente.

Tolerancia a fallos. El manifiesto debe estar disponible antes del corte. La marca de abordado y el pendiente se guardan en una sola transacción local. Si falla el envío o se pierde su respuesta, los registros permanecen y se reenvían con los mismos identificadores; solo se retiran tras la confirmación individual

de Reservas.

Consistencia. Se acepta consistencia eventual entre el aeropuerto y Reservas. Bailis y Ghodsi explican que, si dejan de producirse actualizaciones, las copias terminan convergiendo, pero esa garantía no asegura lecturas actuales durante la desconexión [15]. Por eso, al recuperar la red se intercambian los abordajes pendientes y se revisan los conflictos. El manifiesto puede omitir cambios posteriores a su descarga: se limita al vuelo y su versión, se impiden duplicados locales y se envían discrepancias a revisión del personal. En 5.4 Reservas deduplica por solicitud y por pase del vuelo. Esto no garantiza consistencia global durante F3 ni autoriza vender asientos sin conexión.

Fuentes. [15], [17]

6. Declaración de uso de inteligencia artificial

Debido a que no se realizó una presentación presencial en clase, no se redactaron las preguntas previstas para dicha presentación.

Herramienta	Para qué la usamos	Cómo verificamos
Claude	Apoyo para entender el escenario, revisar el diseño y elaborar la plantilla base de `main.tex`.	Revisión de la correspondencia entre los requisitos del escenario, las decisiones de comunicación y el diagrama; comprobación del formato mediante la compilación del documento.

Referencias

[1] A. D. Birrell y B. J. Nelson, «Implementing Remote Procedure Calls,» *ACM Transactions on Computer Systems*, vol. 2, n.º 1, págs. 39-59, 1984. DOI: 10.1145/2080.357392

[2] M. Niswar, R. A. Safruddin, A. Bustamin e I. Aswad, «Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC,» *International Journal of Electronics and Telecommunications*, vol. 70, n.º 2, págs. 429-436, 2024. DOI: 10.24425/ijet.2024.149562

[3] B. Liskov, L. Shrira y J. Wroclawski, «Efficient At-Most-Once Messages Based on Synchronized Clocks,» *ACM Transactions on Computer Systems*, vol. 9, n.º 2, págs. 125-142, 1991. DOI: 10.1145/103720.103722

[4] P. K. Kumar, R. Agarwal, R. Shivaprasad, D. Sitaram y S. Kalambur, «Performance Characterization of Communication Protocols in Microservice Applications,» en *2021 International Conference on Smart Applications, Communications and Networking (SmartNets)*, IEEE, 2021, págs. 1-5. DOI: 10.1109/SmartNets50376.2021.9555425

[5] P. M. Melliar-Smith y L. E. Moser, «Surviving Network Partitioning,» *Computer*, vol. 31, n.º 3, págs. 62-68, 1998. DOI: 10.1109/2.660191

[6] E. Brewer, «A Certain Freedom: Thoughts on the CAP Theorem,» en *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC '10)*, ACM, 2010, pág. 335. DOI: 10.1145/1835698.1835701

[7] F. Laux y M. Laiho, «SQL Access Patterns for Optimistic Concurrency Control,» en *2009 Computation World: Future Computing, Service Computation, Cognitive, Adaptive, Content, Patterns*, IEEE, 2009. DOI: 10.1109/ComputationWorld.2009.63

[8] C. G. Gray y D. R. Cheriton, «Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency,» en *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP '89)*, ACM, 1989, págs. 202-210. DOI: 10.1145/74850.74870

[9] P. Cao y C. Liu, «Maintaining Strong Cache Consistency in the World Wide Web,» *IEEE Transactions on Computers*, vol. 47, n.º 4, págs. 445-457, 1998. DOI: 10.1109/12.675713

[10] L. Y. Cao y M. T. Özsu, «Evaluation of Strong Consistency Web Caching Techniques,» *World Wide Web*, vol. 5, n.º 2, págs. 95-123, 2002. DOI: 10.1023/A:1019697023170

[11] M. Rostański, K. Grochla y A. Seman, «Evaluation of Highly Available and Fault-Tolerant Middleware Clustered Architectures Using RabbitMQ,» en *Proceedings of the 2014 Federated Conference on Computer Science and Information Systems (FedCSIS)*, IEEE, 2014, págs. 879-884. DOI: 10.15439/2014F48

[12] H. Garcia-Molina y K. Salem, «Sagas,» en *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data (SIGMOD '87)*, ACM, 1987, págs. 249-259. DOI: 10.1145/38713.38742

[13] E. Y. Daraghmi, C.-P. Zhang y S.-M. Yuan, «Enhancing Saga Pattern for Distributed Transactions within a Microservices Architecture,» *Applied Sciences*, vol. 12, n.º 12, pág. 6242, 2022. DOI: 10.3390/app12126242

[14] N. R. Ivaki, F. Araujo y R. Barbosa, «A Middleware for Exactly-Once Semantics in Request-Response Interactions,» en *2012 IEEE 18th Pacific Rim International Symposium on Dependable Computing (PRDC)*, IEEE, 2012, págs. 31-40. DOI: 10.1109/PRDC.2012.23

[15] P. Bailis y A. Ghodsi, «Eventual Consistency Today: Limitations, Extensions, and Beyond,» *ACM Queue*, vol. 11, n.º 3, 2013, **artno** 20. DOI: 10.1145/2460276.2462076

[16] «Transmission Control Protocol (TCP),» RFC Editor, RFC 9293, ago. de 2022. visitado 11 de nov. de 2026. dirección: https://www.rfc-editor.org/rfc/rfc9293

[17] gRPC Authors. «Core concepts, architecture and lifecycle,» visitado 11 de nov. de 2026. dirección: https://grpc.io/docs/what-is-grpc/core-concepts/

[18] gRPC Authors. «About gRPC,» visitado 11 de nov. de 2026. dirección: https://grpc.io/about/

[19] gRPC Authors. «Deadlines,» visitado 11 de nov. de 2026. dirección: https://grpc.io/docs/guides/deadlines/

[20] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed. Amsterdam, The Netherlands: Maarten van Steen, 2023, ISBN: 978-90-815406-3-6.

[21] M. Kleppmann, *Designing Data-Intensive Applications, The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA, USA: O'Reilly Media, 2017, ISBN: 978-1-449-37332-0.

[22] S. Newman, *Building Microservices, Designing Fine-Grained Systems*, 2.ª ed. Sebastopol, CA, USA: O'Reilly Media, 2021, ISBN: 978-1-4920-3402-5.

[23] G. Blinowski, A. Ojdowska y A. Przybylek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803

[24] I. Karabey Aksakalli, T. Çelik, A. B. Can y B. Tekinerdoğan, «Deployment and Communication Patterns in Microservice Architectures: A Systematic Literature Review,» *Journal of Systems and Software*, vol. 180, 2021, **artno** 111014. DOI: 10.1016/j.jss.2021.111014